

Outer and Middle Ear Simulator Code

Uploadable code appendix for the browser-based simulator. The middle-ear implementation follows the supplied class circuit: cavity series block, eardrum-loss shunt, ossicle series block, and joint/cochlear-load shunt branches. The cochlear branch is only a terminating load; inner-ear mechanics are not modeled. The transfer uses an ideal input source and unit-normalized magnitude because the class diagram shows no source resistor or arbitrary gain.

index.html

```
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <title>Outer + Middle Ear Acoustics Lab</title>
    <link rel="stylesheet" href="styles.css" />
  </head>
  <body>
    <div class="page-shell">
      <header class="hero panel">
        <div class="hero__copy">
          <p class="eyebrow">Outer + Middle Ear Acoustics Lab</p>
          <h1>Interactive anatomy-to-cochlear-load transfer model</h1>
          <p class="hero__lede">
            Follow the signal from pinna and concha through the ear canal,
            eardrum, ossicles, and cochlear load with adjustable acoustic and
            mechanical parameters.
          </p>
        </div>
        <div class="hero__stats">
          <div class="stat-card">
            <span class="stat-card__label">Model span</span>
            <strong>20 Hz to 20 kHz</strong>
          </div>
          <div class="stat-card">
            <span class="stat-card__label">Signal path</span>
            <strong>Pinna to cochlea</strong>
          </div>
          <div class="stat-card">
            <span class="stat-card__label">Preview mode</span>
            <strong>Live plots + audio</strong>
          </div>
        </div>
      </header>

      <main class="layout">
        <aside class="panel controls">
          <div class="section-heading">
            <h2>Biological Controls</h2>
            <p>Adjust the anatomy and watch the transfer function update.</p>
          </div>

          <section class="control-group">
            <div class="control-group__heading">
              <h3>Ear Canal</h3>
              <p>Quarter-wave modes and simplified IEC 318 branches.</p>
            </div>

            <label class="control">
              <div class="control__top">
                <span>Canal length</span>
                <output id="canalLengthMmValue">27.0 mm</output>
              </div>
              <input
                type="range"
                min="20"
                max="40"
                step="0.5"
                value="27"
                data-param="canalLengthMm"
              />
            </label>

            <label class="control">
              <div class="control__top">
                <span>Canal diameter</span>
                <output id="canalDiameterMmValue">7.0 mm</output>
              </div>
              <input
                type="range"
                min="4"
                max="10"
                step="0.1"
                value="7"
              />
            </label>
          </section>
        </aside>
      </main>
    </div>
  </body>
</html>
```

```

    data-param="canalDiameterMm"
  />
</label>

<label class="toggle">
  <input type="checkbox" id="iec318Enabled" checked />
  <span>Include simplified IEC 318 equivalent-circuit branches</span>
</label>
</section>

<section class="control-group control-group--middle">
  <div class="control-group__heading">
    <h3>Middle Ear</h3>
    <p>Middle-ear lumped impedances from the class circuit; the cochlear branch is used only as the terminating load.</p>
  </div>

  <label class="control">
    <div class="control__top">
      <span>Eardrum + ossicle inertance</span>
      <output id="middleInertanceValue">0.0012 m.u.</output>
    </div>
    <input
      type="range"
      min="0.0004"
      max="0.0030"
      step="0.0001"
      value="0.0012"
      data-param="middleInertance"
    />
  </label>

  <label class="control">
    <div class="control__top">
      <span>Eardrum compliance</span>
      <output id="middleComplianceValue">0.000020 m.u.</output>
    </div>
    <input
      type="range"
      min="0.000008"
      max="0.000060"
      step="0.000001"
      value="0.000020"
      data-param="middleCompliance"
    />
  </label>

  <label class="control">
    <div class="control__top">
      <span>Eardrum/joint resistance</span>
      <output id="middleResistanceValue">0.65 m.u.</output>
    </div>
    <input
      type="range"
      min="0.10"
      max="2.00"
      step="0.05"
      value="0.65"
      data-param="middleResistance"
    />
  </label>

  <label class="control">
    <div class="control__top">
      <span>Middle-ear cavity volume</span>
      <output id="middleCavityVolumeCcValue">1.5 cc</output>
    </div>
    <input
      type="range"
      min="0.5"
      max="3.0"
      step="0.1"
      value="1.5"
      data-param="middleCavityVolumeCc"
    />
  </label>

  <label class="control">
    <div class="control__top">

```

```

    <span>Incudo-stapedial compliance</span>
    <output id="jointComplianceValue">0.000035 m.u.</output>
  </div>
  <input
    type="range"
    min="0.000012"
    max="0.000080"
    step="0.000001"
    value="0.000035"
    data-param="jointCompliance"
  />
</label>

<label class="control">
  <div class="control__top">
    <span>Cochlear load resistance</span>
    <output id="cochlearLoadValue">1.80 m.u.</output>
  </div>
  <input
    type="range"
    min="0.50"
    max="4.00"
    step="0.10"
    value="1.80"
    data-param="cochlearLoad"
  />
</label>

<p class="microcopy topology-note">
  Class topology: cavity series block, eardrum-loss shunt, ossicle
  series block, then joint and cochlear-load shunt branches.
</p>
</section>

<section class="control-group">
  <div class="control-group__heading">
    <h3>Concha Cavity</h3>
    <p>Helmholtz resonance from volume and aperture geometry.</p>
  </div>

  <label class="control">
    <div class="control__top">
      <span>Concha depth</span>
      <output id="conchaDepthMmValue">12.0 mm</output>
    </div>
    <input
      type="range"
      min="5"
      max="20"
      step="0.5"
      value="12"
      data-param="conchaDepthMm"
    />
  </label>

  <label class="control">
    <div class="control__top">
      <span>Concha volume</span>
      <output id="conchaVolumeCcValue">3.0 cc</output>
    </div>
    <input
      type="range"
      min="1"
      max="8"
      step="0.1"
      value="3"
      data-param="conchaVolumeCc"
    />
  </label>

  <label class="control">
    <div class="control__top">
      <span>Aperture radius</span>
      <output id="conchaApertureMmValue">4.5 mm</output>
    </div>
    <input
      type="range"
      min="2"

```

```
max="8"
step="0.1"
value="4.5"
data-param="conchaApertureMm"
/>
</label>
</section>
```

```
<section class="control-group">
<div class="control-group__heading">
<h3>Concha Reflector</h3>
<p>Parabolic-reflector approximation with a 2 kHz-ish break.</p>
</div>
```

```
<label class="control">
<div class="control__top">
<span>Reflector length L</span>
<output id="reflectorLengthMmValue">5.0 mm</output>
</div>
<input
type="range"
min="2"
max="12"
step="0.2"
value="5"
data-param="reflectorLengthMm"
/>
</label>
```

```
<label class="control">
<div class="control__top">
<span>Reflector radius a</span>
<output id="reflectorRadiusMmValue">10.0 mm</output>
</div>
<input
type="range"
min="5"
max="18"
step="0.2"
value="10"
data-param="reflectorRadiusMm"
/>
</label>
</section>
```

```
<section class="control-group">
<div class="control-group__heading">
<h3>Pinna Reflection</h3>
<p>Direction-dependent path difference and destructive interference.</p>
</div>
```

```
<label class="control">
<div class="control__top">
<span>Base reflection path</span>
<output id="reflectionPathMmValue">14.0 mm</output>
</div>
<input
type="range"
min="8"
max="24"
step="0.2"
value="14"
data-param="reflectionPathMm"
/>
</label>
```

```
<label class="control">
<div class="control__top">
<span>Reflection strength</span>
<output id="reflectionStrengthValue">0.60</output>
</div>
<input
type="range"
min="0.05"
max="0.95"
step="0.01"
value="0.6"
data-param="reflectionStrength"
/>
```

```

/>
</label>

<label class="control">
  <div class="control__top">
    <span>Source azimuth</span>
    <output id="azimuthDegValue">0 deg</output>
  </div>
  <input
    type="range"
    min="-90"
    max="90"
    step="1"
    value="0"
    data-param="azimuthDeg"
  />
</label>

<label class="control">
  <div class="control__top">
    <span>Source elevation</span>
    <output id="elevationDegValue">0 deg</output>
  </div>
  <input
    type="range"
    min="-45"
    max="90"
    step="1"
    value="0"
    data-param="elevationDeg"
  />
</label>
</section>

<section class="control-group">
  <div class="control-group__heading">
    <h3>Actions</h3>
    <p>Reset values or export a report-ready summary snippet.</p>
  </div>

  <div class="button-row">
    <button id="resetButton" type="button">Reset parameters</button>
    <button id="exportButton" type="button">Download report snippet</button>
  </div>
</section>
</aside>

<section class="plots">
  <div class="panel plot-card plot-card--wide">
    <div class="plot-card__header">
      <div>
        <h2>Outer, Middle, and Combined Response</h2>
        <p>Outer-ear transfer cascaded with the middle-ear transmission model.</p>
      </div>
      <div class="summary-chip" id="summaryChip">Calculating...</div>
    </div>
    <svg
      id="combinedPlot"
      class="plot"
      viewBox="0 0 760 280"
      role="img"
      aria-label="Combined frequency response plot"
    ></svg>
  </div>

  <div class="plot-grid middle-plot-grid">
    <div class="panel plot-card">
      <div class="plot-card__header plot-card__header--compact">
        <div>
          <h3>Middle-Ear Transmission</h3>
          <p>Magnitude from the middle-ear input to the cochlear load.</p>
        </div>
        </div>
        <svg
          id="middlePlot"
          class="plot plot--small"
          viewBox="0 0 360 220"
          role="img"

```

```
    aria-label="Middle-ear transmission magnitude plot"
  ></svg>
</div>
```

```
<div class="panel plot-card">
  <div class="plot-card__header plot-card__header--compact">
    <div>
      <h3>Middle-Ear Transmission Phase</h3>
      <p>Phase of the modeled pressure transfer function.</p>
    </div>
  </div>
  <svg
    id="middlePhasePlot"
    class="plot plot--small"
    viewBox="0 0 360 220"
    role="img"
    aria-label="Middle-ear transmission phase plot"
  ></svg>
</div>
```

```
<div class="panel plot-card">
  <div class="plot-card__header plot-card__header--compact">
    <div>
      <h3>Middle-Ear Input Impedance</h3>
      <p>Magnitude of the input impedance in model units.</p>
    </div>
  </div>
  <svg
    id="middleImpedancePlot"
    class="plot plot--small"
    viewBox="0 0 360 220"
    role="img"
    aria-label="Middle-ear input impedance plot"
  ></svg>
</div>
</div>
```

```
<div class="plot-grid">
  <div class="panel plot-card">
    <div class="plot-card__header plot-card__header--compact">
      <div>
        <h3>Concha Reflector</h3>
        <p>Parabolic gain rise above the break frequency.</p>
      </div>
    </div>
    <svg
      id="reflectorPlot"
      class="plot plot--small"
      viewBox="0 0 360 220"
      role="img"
      aria-label="Concha reflector response plot"
    ></svg>
  </div>
</div>
```

```
<div class="panel plot-card">
  <div class="plot-card__header plot-card__header--compact">
    <div>
      <h3>Concha Cavity</h3>
      <p>Helmholtz-like resonance from volume and opening.</p>
    </div>
  </div>
  <svg
    id="conchaPlot"
    class="plot plot--small"
    viewBox="0 0 360 220"
    role="img"
    aria-label="Concha cavity response plot"
  ></svg>
</div>
```

```
<div class="panel plot-card">
  <div class="plot-card__header plot-card__header--compact">
    <div>
      <h3>Ear Canal</h3>
      <p>Quarter-wave resonances with optional IEC 318 branches.</p>
    </div>
  </div>
  <svg
```

```

    id="canalPlot"
    class="plot plot--small"
    viewBox="0 0 360 220"
    role="img"
    aria-label="Ear canal response plot"
  ></svg>
</div>

<div class="panel plot-card">
  <div class="plot-card__header plot-card__header--compact">
    <div>
      <h3>Pinna Reflection</h3>
      <p>Comb-style interference from delayed reflected sound.</p>
    </div>
  </div>
  <svg
    id="reflectionPlot"
    class="plot plot--small"
    viewBox="0 0 360 220"
    role="img"
    aria-label="Pinna reflection response plot"
  ></svg>
</div>
</div>

<div class="panel plot-card plot-card--wide">
  <div class="plot-card__header sweep-header">
    <div>
      <h2>Real-Time Parameter Sweep</h2>
      <p>Overlay combined responses across the selected parameter range.</p>
    </div>
    <div class="sweep-controls">
      <label class="select-control">
        <span>Swept parameter</span>
        <select id="sweepParameter"></select>
      </label>
      <label class="select-control">
        <span>Trace count</span>
        <select id="sweepTraceCount">
          <option value="4">4</option>
          <option value="5">5</option>
          <option value="6">6</option>
          <option value="7" selected>7</option>
          <option value="8">8</option>
          <option value="9">9</option>
        </select>
      </label>
    </div>
  </div>
  <svg
    id="sweepPlot"
    class="plot"
    viewBox="0 0 760 280"
    role="img"
    aria-label="Parameter sweep plot"
  ></svg>
</div>

<div class="results-grid">
  <div class="panel result-card">
    <h2>Calculated Markers</h2>
    <div id="markerList" class="marker-list"></div>
  </div>

  <div class="panel result-card">
    <h2>AI-Assisted Experiments</h2>
    <p>Each experiment changes one required parameter by +20% from the current baseline.</p>
    <div id="experimentList" class="experiment-list"></div>
  </div>

  <div class="panel result-card">
    <h2>Model Equations</h2>
    <div class="equation-list">
      <p><code>f_break = c / (16a)</code></p>
      <p><code>f_H = c / (2pi) * sqrt(A / (V L_eff))</code></p>
      <p><code>f_n = n c / (4 L_eff)</code></p>
      <p><code>f_notch = c / (2 r_eff)</code></p>
      <p><code>Z_R = R, Z_L = jωL, Z_C = 1/(jωC)</code></p>
    </div>
  </div>

```

`Z_cavity = (Z_CP + Z_LA + R_A) || R_M || Z_CT`

`Z_eardrum = Z_CD1 + (Z_LD || (Z_CD2 + R_D2)) + R_D1`

`Z_distal = Z_joint || Z_cochlea`

`Z_in = Z_cavity + (Z_eardrum || (Z_ossicles + Z_distal))`

`H_total = H_outer × H_middle`

These equations follow the class notes and use a simplified damping model so the plots stay finite and interactive.

<div class="panel result-card">

<h2>Audio Preview</h2>

<div class="audio-controls">

<div class="button-row">

<button id="playNoiseButton" type="button">Play noise</button>

<button id="playSweepButton" type="button">Play sweep</button>

<button id="stopAudioButton" type="button">Stop</button>

</div>

<label class="file-control">

Upload audio

<input id="audioFileInput" type="file" accept="audio/*" />

</label>

<div class="button-row">

<button id="playFileButton" type="button" disabled>

Play uploaded audio

</button>

</div>

</div>

<p class="microcopy">

Audio uses a lightweight Web Audio approximation of the plotted model so you can hear the current anatomy settings.

</p>

<div id="audioStatus" class="audio-status">Audio idle.</div>

</div>

</section>

</main>

</div>

<script src="app.js?v=middle-ear-1"></script>

</body>

</html>

styles.css

```
:root {
  --bg: #f6efc3;
  --bg-strong: #ead8c0;
  --panel: rgba(255, 250, 242, 0.86);
  --panel-strong: rgba(255, 249, 240, 0.95);
  --line: rgba(94, 66, 43, 0.16);
  --ink: #2d2016;
  --muted: #715644;
  --accent: #d5673f;
  --accent-deep: #b44c2d;
  --accent-soft: rgba(213, 103, 63, 0.16);
  --teal: #2a7f7a;
  --teal-soft: rgba(42, 127, 122, 0.18);
  --gold: #c4902f;
  --plot-grid: rgba(45, 32, 22, 0.12);
  --shadow: 0 18px 50px rgba(73, 47, 27, 0.12);
  --radius-lg: 28px;
  --radius-md: 18px;
  --radius-sm: 12px;
  --font-body: "Avenir Next", "Trebuchet MS", "Segoe UI", sans-serif;
  --font-display: "Iowan Old Style", "Palatino Linotype", "Book Antiqua", serif;
}

* {
  box-sizing: border-box;
}

body {
  margin: 0;
  min-height: 100vh;
  font-family: var(--font-body);
  color: var(--ink);
  background:
    radial-gradient(circle at top left, rgba(212, 131, 80, 0.26), transparent 34%),
    radial-gradient(circle at top right, rgba(42, 127, 122, 0.18), transparent 28%),
    linear-gradient(160deg, #f9f4ed 0%, var(--bg) 52%, var(--bg-strong) 100%);
}

body::before,
body::after {
  content: "";
  position: fixed;
  inset: auto;
  width: 28rem;
  height: 28rem;
  border-radius: 999px;
  filter: blur(24px);
  opacity: 0.4;
  pointer-events: none;
}

body::before {
  top: -9rem;
  left: -8rem;
  background: radial-gradient(circle, rgba(213, 103, 63, 0.22), transparent 68%);
}

body::after {
  right: -10rem;
  bottom: -9rem;
  background: radial-gradient(circle, rgba(42, 127, 122, 0.18), transparent 70%);
}

.page-shell {
  width: min(1440px, calc(100vw - 2rem));
  margin: 1rem auto 2.5rem;
  position: relative;
}

.panel {
  background: var(--panel);
  border: 1px solid rgba(255, 255, 255, 0.68);
  border-radius: var(--radius-lg);
  box-shadow: var(--shadow);
  backdrop-filter: blur(18px);
}
```

```

}

.hero {
  display: grid;
  grid-template-columns: minmax(0, 1.6fr) minmax(280px, 0.9fr);
  gap: 1.5rem;
  padding: 2rem;
  overflow: hidden;
  position: relative;
}

.hero::after {
  content: "";
  position: absolute;
  right: 2rem;
  top: 1.75rem;
  width: 11rem;
  height: 15rem;
  border-radius: 58% 42% 52% 48% / 42% 44% 56% 58%;
  background:
    radial-gradient(circle at 34% 34%, rgba(255, 255, 255, 0.9), transparent 24%),
    linear-gradient(160deg, rgba(213, 103, 63, 0.12), rgba(42, 127, 122, 0.16));
  border: 1px solid rgba(113, 86, 68, 0.12);
}

.eyebrow {
  margin: 0 0 0.65rem;
  letter-spacing: 0.18em;
  text-transform: uppercase;
  font-size: 0.75rem;
  color: var(--accent-deep);
  font-weight: 700;
}

.hero h1 {
  margin: 0;
  font-family: var(--font-display);
  font-size: clamp(2.2rem, 3.8vw, 4rem);
  line-height: 0.95;
  max-width: 13ch;
}

.hero__lede {
  max-width: 58ch;
  margin: 1rem 0 0;
  color: var(--muted);
  font-size: 1rem;
  line-height: 1.65;
}

.hero__stats {
  display: grid;
  align-content: end;
  gap: 0.85rem;
  position: relative;
  z-index: 1;
}

.stat-card {
  background: var(--panel-strong);
  border: 1px solid rgba(113, 86, 68, 0.1);
  border-radius: var(--radius-md);
  padding: 1rem 1.1rem;
}

.stat-card__label {
  display: block;
  font-size: 0.78rem;
  text-transform: uppercase;
  letter-spacing: 0.12em;
  color: var(--muted);
  margin-bottom: 0.35rem;
}

.stat-card strong {
  font-size: 1rem;
}

```

```

.layout {
  display: grid;
  grid-template-columns: minmax(320px, 360px) minmax(0, 1fr);
  gap: 1rem;
  margin-top: 1rem;
  align-items: start;
}

.controls {
  padding: 1.25rem;
  position: sticky;
  top: 1rem;
}

.section-heading h2,
.plot-card h2,
.result-card h2,
.control-group h3,
.plot-card h3 {
  margin: 0;
  font-family: var(--font-display);
  font-weight: 700;
}

.section-heading p,
.control-group__heading p,
.plot-card p,
.result-card p {
  margin: 0.35rem 0 0;
  color: var(--muted);
  line-height: 1.55;
}

.control-group {
  margin-top: 1.2rem;
  padding: 1rem;
  background: rgba(255, 255, 255, 0.48);
  border-radius: var(--radius-md);
  border: 1px solid rgba(113, 86, 68, 0.08);
}

.control-group--middle {
  background:
    linear-gradient(145deg, rgba(139, 91, 157, 0.1), rgba(255, 255, 255, 0.48));
  border-color: rgba(139, 91, 157, 0.2);
}

.topology-note {
  padding-top: 0.65rem;
  border-top: 1px solid rgba(113, 86, 68, 0.12);
}

.control-group__heading {
  margin-bottom: 0.8rem;
}

.control + .control,
.toggle + .control,
.control + .toggle {
  margin-top: 0.9rem;
}

.control {
  display: block;
}

.control__top {
  display: flex;
  justify-content: space-between;
  gap: 0.75rem;
  font-size: 0.95rem;
  margin-bottom: 0.45rem;
}

.control output {
  font-weight: 700;
  color: var(--accent-deep);
}

```

```

input[type="range"] {
  width: 100%;
  margin: 0;
  accent-color: var(--accent);
}

.toggle {
  display: flex;
  gap: 0.75rem;
  align-items: start;
  font-size: 0.93rem;
  color: var(--ink);
}

.toggle input {
  margin-top: 0.1rem;
  accent-color: var(--teal);
}

.button-row {
  display: flex;
  flex-wrap: wrap;
  gap: 0.7rem;
}

button,
select,
input[type="file"] {
  font: inherit;
}

button {
  border: 0;
  border-radius: 999px;
  padding: 0.78rem 1.1rem;
  background: linear-gradient(135deg, var(--accent), var(--accent-deep));
  color: white;
  font-weight: 700;
  cursor: pointer;
  box-shadow: 0 10px 24px rgba(180, 76, 45, 0.18);
  transition: transform 150ms ease, box-shadow 150ms ease, opacity 150ms ease;
}

button:hover {
  transform: translateY(-1px);
  box-shadow: 0 14px 28px rgba(180, 76, 45, 0.22);
}

button:disabled {
  cursor: not-allowed;
  opacity: 0.45;
  box-shadow: none;
}

.plots {
  display: grid;
  gap: 1rem;
}

.plot-card {
  padding: 1rem;
}

.plot-card--wide {
  padding: 1.1rem 1.15rem 0.95rem;
}

.plot-card__header {
  display: flex;
  justify-content: space-between;
  gap: 1rem;
  align-items: start;
  margin-bottom: 0.8rem;
}

.plot-card__header--compact {
  margin-bottom: 0.55rem;
}

```

```

}

.summary-chip {
  padding: 0.6rem 0.85rem;
  border-radius: 999px;
  background: var(--accent-soft);
  color: var(--accent-deep);
  font-weight: 700;
  font-size: 0.88rem;
  align-self: center;
}

.plot-grid {
  display: grid;
  grid-template-columns: repeat(2, minmax(0, 1fr));
  gap: 1rem;
}

.middle-plot-grid {
  grid-template-columns: repeat(3, minmax(0, 1fr));
}

.plot {
  width: 100%;
  display: block;
  border-radius: 16px;
  background:
    linear-gradient(180deg, rgba(255, 255, 255, 0.66), rgba(255, 255, 255, 0.18)),
    linear-gradient(180deg, rgba(42, 127, 122, 0.03), rgba(213, 103, 63, 0.04));
}

.plot--small {
  min-height: 220px;
}

.sweep-header {
  align-items: end;
}

.sweep-controls {
  display: flex;
  flex-wrap: wrap;
  gap: 0.85rem;
}

.select-control {
  display: grid;
  gap: 0.35rem;
  min-width: 170px;
  font-size: 0.88rem;
}

.select-control span {
  color: var(--muted);
}

select,
.file-control input[type="file"] {
  border: 1px solid rgba(113, 86, 68, 0.18);
  border-radius: 12px;
  background: rgba(255, 255, 255, 0.9);
  padding: 0.7rem 0.8rem;
}

.results-grid {
  display: grid;
  grid-template-columns: repeat(3, minmax(0, 1fr));
  gap: 1rem;
}

.result-card {
  padding: 1.1rem;
}

.marker-list {
  display: grid;
  gap: 0.7rem;
  margin-top: 0.9rem;
}

```

```

}

.marker-row {
  display: flex;
  justify-content: space-between;
  gap: 0.75rem;
  padding: 0.78rem 0.9rem;
  border-radius: 14px;
  background: rgba(255, 255, 255, 0.55);
  border: 1px solid rgba(113, 86, 68, 0.08);
}

.marker-row strong {
  font-size: 0.95rem;
}

.marker-row span:last-child {
  font-weight: 700;
  color: var(--teal);
}

.experiment-list {
  display: grid;
  gap: 0.75rem;
  margin-top: 0.9rem;
}

.experiment-row {
  padding: 0.85rem 0.9rem;
  border-radius: 14px;
  background: rgba(139, 91, 157, 0.08);
  border: 1px solid rgba(139, 91, 157, 0.14);
}

.experiment-row strong {
  color: #6f427f;
}

.experiment-row p {
  margin: 0.45rem 0 0;
  font-size: 0.86rem;
  line-height: 1.45;
}

.experiment-row p span {
  font-weight: 700;
}

.equation-list {
  display: grid;
  gap: 0.55rem;
  margin-top: 0.95rem;
}

.equation-list p {
  margin: 0;
  padding: 0.75rem 0.85rem;
  border-radius: 14px;
  background: var(--teal-soft);
  color: var(--ink);
}

code {
  font-family: "SFMono-Regular", "Menlo", "Consolas", monospace;
  font-size: 0.92rem;
}

.microcopy {
  font-size: 0.9rem;
}

.audio-controls {
  display: grid;
  gap: 0.8rem;
  margin-top: 0.9rem;
}

.file-control {

```

```

display: grid;
gap: 0.35rem;
font-size: 0.88rem;
color: var(--muted);
}

.audio-status {
margin-top: 0.75rem;
padding: 0.85rem 0.95rem;
border-radius: 14px;
background: rgba(42, 127, 122, 0.08);
color: var(--teal);
font-weight: 700;
}

.axis-label,
.tick-label,
.legend-label,
.marker-label {
font-family: var(--font-body);
fill: var(--muted);
}

.axis-label {
font-size: 12px;
font-weight: 700;
}

.tick-label {
font-size: 11px;
}

.legend-label {
font-size: 11px;
font-weight: 700;
}

.marker-label {
font-size: 11px;
}

@media (max-width: 1120px) {
.layout {
grid-template-columns: 1fr;
}

.controls {
position: static;
}

.results-grid {
grid-template-columns: 1fr;
}
}

@media (max-width: 820px) {
.page-shell {
width: min(100vw, calc(100vw - 1rem));
margin: 0.5rem auto 2rem;
}

.hero {
grid-template-columns: 1fr;
padding: 1.4rem;
}

.hero::after {
width: 8.2rem;
height: 11rem;
right: 1rem;
top: 1rem;
}

.plot-grid {
grid-template-columns: 1fr;
}

.middle-plot-grid {

```

```
    grid-template-columns: 1fr;
  }

  .plot-card__header,
  .sweep-header {
    flex-direction: column;
  }

  .sweep-controls {
    width: 100%;
  }

  .select-control {
    min-width: 0;
    width: 100%;
  }
}
```

app.js

```
const SPEED_OF_SOUND = 343;
const ANALYSIS_SAMPLE_RATE = 96 000;
const FREQUENCIES = logSpace(20, 20 000, 480);
const DB_RANGE = { min: -18, max: 24 };
```

```
const PARAMETER_DEFS = [
  {
    key: "canalLengthMm",
    label: "Canal length",
    unit: "mm",
    min: 20,
    max: 40,
    step: 0.5,
    defaultValue: 27,
  },
  {
    key: "canalDiameterMm",
    label: "Canal diameter",
    unit: "mm",
    min: 4,
    max: 10,
    step: 0.1,
    defaultValue: 7,
  },
  {
    key: "conchaDepthMm",
    label: "Concha depth",
    unit: "mm",
    min: 5,
    max: 20,
    step: 0.5,
    defaultValue: 12,
  },
  {
    key: "conchaVolumeCc",
    label: "Concha volume",
    unit: "cc",
    min: 1,
    max: 8,
    step: 0.1,
    defaultValue: 3,
  },
  {
    key: "conchaApertureMm",
    label: "Aperture radius",
    unit: "mm",
    min: 2,
    max: 8,
    step: 0.1,
    defaultValue: 4.5,
  },
  {
    key: "reflectorLengthMm",
    label: "Reflector length L",
    unit: "mm",
    min: 2,
    max: 12,
    step: 0.2,
    defaultValue: 5,
  },
  {
    key: "reflectorRadiusMm",
    label: "Reflector radius a",
    unit: "mm",
    min: 5,
    max: 18,
    step: 0.2,
    defaultValue: 10,
  },
  {
    key: "reflectionPathMm",
    label: "Base reflection path",
    unit: "mm",
    min: 8,
    max: 24,
```

```

    step: 0.2,
    defaultValue: 14,
  },
  {
    key: "reflectionStrength",
    label: "Reflection strength",
    unit: "",
    min: 0.05,
    max: 0.95,
    step: 0.01,
    defaultValue: 0.6,
  },
  {
    key: "azimuthDeg",
    label: "Source azimuth",
    unit: "deg",
    min: -90,
    max: 90,
    step: 1,
    defaultValue: 0,
  },
  {
    key: "elevationDeg",
    label: "Source elevation",
    unit: "deg",
    min: -45,
    max: 90,
    step: 1,
    defaultValue: 0,
  },
  {
    key: "middleInertance",
    label: "Eardrum + ossicle inertance",
    unit: "m.u.",
    min: 0.0004,
    max: 0.003,
    step: 0.0001,
    defaultValue: 0.0012,
  },
  {
    key: "middleCompliance",
    label: "Eardrum compliance",
    unit: "m.u.",
    min: 0.000008,
    max: 0.00006,
    step: 0.000001,
    defaultValue: 0.00002,
  },
  {
    key: "middleResistance",
    label: "Eardrum/joint resistance",
    unit: "m.u.",
    min: 0.1,
    max: 2,
    step: 0.05,
    defaultValue: 0.65,
  },
  {
    key: "middleCavityVolumeCc",
    label: "Middle-ear cavity volume",
    unit: "cc",
    min: 0.5,
    max: 3,
    step: 0.1,
    defaultValue: 1.5,
  },
  {
    key: "jointCompliance",
    label: "Incudo-stapedial compliance",
    unit: "m.u.",
    min: 0.000012,
    max: 0.00008,
    step: 0.000001,
    defaultValue: 0.000035,
  },
  {
    key: "cochlearLoad",
    label: "Cochlear load resistance",

```

```

    unit: "m.u.",
    min: 0.5,
    max: 4,
    step: 0.1,
    defaultValue: 1.8,
  },
];

const PARAMETER_LOOKUP = Object.fromEntries(
  PARAMETER_DEFS.map((definition) => [definition.key, definition]),
);

const DEFAULTS = Object.fromEntries(
  PARAMETER_DEFS.map((definition) => [definition.key, definition.defaultValue]),
);

const SWEEPABLE_KEYS = [
  "canalLengthMm",
  "canalDiameterMm",
  "conchaDepthMm",
  "conchaVolumeCc",
  "conchaApertureMm",
  "reflectorLengthMm",
  "reflectorRadiusMm",
  "reflectionPathMm",
  "reflectionStrength",
  "azimuthDeg",
  "elevationDeg",
  "middleInertance",
  "middleCompliance",
  "middleResistance",
  "middleCavityVolumeCc",
  "jointCompliance",
  "cochlearLoad",
];

const COLORS = {
  outer: "#2a7f7a",
  middle: "#8b5b9d",
  total: "#d5673f",
  combined: "#d5673f",
  reflector: "#c4902f",
  concha: "#2a7f7a",
  canal: "#2f5f9d",
  reflection: "#b44c2d",
};

const MIDDLE_CIRCUIT_CONSTANTS = {
  cavityPassageCompliance: 0.000018,
  cavityPassageInertance: 0.0007,
  cavityPassageResistance: 0.08,
  cavityWallResistance: 0.18,
  eardrumSecondaryCompliance: 0.000035,
  eardrumSecondaryResistance: 0.28,
  cochlearCompliance: 0.00004,
  cochlearInertance: 0.00006,
  // The class diagram shows no source resistor, so use an ideal pressure source.
  sourceImpedance: 0,
  // Keep the transmission as a unit-normalized circuit ratio.
  transmissionScale: 1,
};

const dom = {
  parameterInputs: Array.from(document.querySelectorAll("[data-param]")),
  iecToggle: document.getElementById("iec318Enabled"),
  resetButton: document.getElementById("resetButton"),
  exportButton: document.getElementById("exportButton"),
  sweepParameter: document.getElementById("sweepParameter"),
  sweepTraceCount: document.getElementById("sweepTraceCount"),
  markerList: document.getElementById("markerList"),
  summaryChip: document.getElementById("summaryChip"),
  audioStatus: document.getElementById("audioStatus"),
  audioFileInput: document.getElementById("audioFileInput"),
  playNoiseButton: document.getElementById("playNoiseButton"),
  playSweepButton: document.getElementById("playSweepButton"),
  playFileButton: document.getElementById("playFileButton"),
  stopAudioButton: document.getElementById("stopAudioButton"),
  plots: {

```

```

combined: document.getElementById("combinedPlot"),
reflector: document.getElementById("reflectorPlot"),
concha: document.getElementById("conchaPlot"),
canal: document.getElementById("canalPlot"),
reflection: document.getElementById("reflectionPlot"),
middle: document.getElementById("middlePlot"),
middlePhase: document.getElementById("middlePhasePlot"),
middleImpedance: document.getElementById("middleImpedancePlot"),
sweep: document.getElementById("sweepPlot"),
},
experimentList: document.getElementById("experimentList"),
};

const audioState = {
  context: null,
  source: null,
  sourceNodes: [],
  activeNodes: [],
  uploadedBuffer: null,
};

let latestModel = null;

initialize();

function initialize() {
  buildSweepMenu();
  bindParameterEvents();
  bindActionEvents();
  refreshParameterReadouts();
  updateView();
}

function buildSweepMenu() {
  SWEEPABLE_KEYS.forEach((key) => {
    const option = document.createElement("option");
    option.value = key;
    option.textContent = PARAMETER_LOOKUP[key].label;
    dom.sweepParameter.appendChild(option);
  });
  dom.sweepParameter.value = "canalLengthMm";
}

function bindParameterEvents() {
  dom.parameterInputs.forEach((input) => {
    input.addEventListener("input", () => {
      updateReadout(input.dataset.param, Number(input.value));
      updateView();
    });
  });

  dom.iecToggle.addEventListener("change", updateView);
  dom.sweepParameter.addEventListener("change", updateView);
  dom.sweepTraceCount.addEventListener("change", updateView);
}

function bindActionEvents() {
  dom.resetButton.addEventListener("click", resetParameters);
  dom.exportButton.addEventListener("click", exportReportSnippet);
  dom.playNoiseButton.addEventListener("click", playNoisePreview);
  dom.playSweepButton.addEventListener("click", playToneSweepPreview);
  dom.stopAudioButton.addEventListener("click", stopAudio);
  dom.playFileButton.addEventListener("click", playUploadedAudio);
  dom.audioFileInput.addEventListener("change", handleAudioUpload);
}

function refreshParameterReadouts() {
  PARAMETER_DEFS.forEach((definition) => {
    const input = getInput(definition.key);
    updateReadout(definition.key, Number(input.value));
  });
}

function resetParameters() {
  PARAMETER_DEFS.forEach((definition) => {
    const input = getInput(definition.key);
    input.value = definition.defaultValue;
  });
}

```

```

dom.iecToggle.checked = true;
refreshParameterReadouts();
updateView();
setAudioStatus("Parameters reset to the baseline anatomy.");
}

function updateReadout(key, value) {
  const output = document.getElementById(`${key}Value`);
  if (output) {
    output.textContent = formatParameterValue(key, value);
  }
}

function readParameters() {
  const parameters = {};
  PARAMETER_DEFS.forEach((definition) => {
    parameters[definition.key] = Number(getInput(definition.key).value);
  });
  parameters.iec318Enabled = dom.iecToggle.checked;
  return parameters;
}

function updateView() {
  const parameters = readParameters();
  const model = computeModel(parameters);
  latestModel = model;

  renderPlot(dom.plots.combined, [
    { label: "Outer ear", values: model.outerDb, color: COLORS.outer },
    { label: "Middle ear", values: model.middleDb, color: COLORS.middle },
    { label: "Combined", values: model.combinedTotalDb, color: COLORS.total },
  ], {
    yRange: DB_RANGE,
    markers: [
      {
        frequency: model.metrics.reflector.breakHz,
        label: "Reflector break",
      },
      {
        frequency: model.metrics.concha.helmholtzHz,
        label: "Helmholtz",
      },
      {
        frequency: model.metrics.canal.f1,
        label: "Canal f1",
      },
      {
        frequency: model.metrics.reflection.notchHz,
        label: "Pinna notch",
      },
    ],
  });

  renderPlot(dom.plots.reflector, [
    { label: "Reflector", values: model.reflectorDb, color: COLORS.reflector },
  ], { yRange: DB_RANGE });

  renderPlot(dom.plots.concha, [
    { label: "Concha", values: model.conchaDb, color: COLORS.concha },
  ], { yRange: DB_RANGE });

  renderPlot(dom.plots.canal, [
    { label: "Canal", values: model.canalDb, color: COLORS.canal },
  ], { yRange: DB_RANGE });

  renderPlot(dom.plots.reflection, [
    {
      label: "Reflection",
      values: model.reflectionDb,
      color: COLORS.reflection,
    },
  ], { yRange: DB_RANGE });

  renderPlot(dom.plots.middle, [
    { label: "Middle ear", values: model.middleDb, color: COLORS.middle },
  ], {
    yRange: { min: -36, max: 12 },
    yTicks: [12, 6, 0, -6, -12, -18, -24, -30, -36],
  });

```

```

markers: [
  {
    frequency: model.metrics.middle.transmissionPeak.frequency,
    label: "Transmission peak",
  },
],
});

renderPlot(dom.plots.middlePhase, [
  { label: "Phase", values: model.middlePhaseDeg, color: COLORS.middle },
], {
  yRange: { min: -180, max: 180 },
  yLabel: "Phase (degrees)",
  yTicks: [-180, -120, -60, 0, 60, 120, 180],
});

renderPlot(dom.plots.middleImpedance, [
  {
    label: "Input impedance",
    values: model.middleInputImpedanceDb,
    color: COLORS.canal,
  },
], {
  yRange: { min: -12, max: 30 },
  yTicks: [30, 24, 18, 12, 6, 0, -6, -12],
});

renderPlot(dom.plots.sweep, buildSweepSeries(parameters), {
  yRange: DB_RANGE,
  showLegend: true,
});

renderMarkerList(model);
renderExperiments(parameters, model);
renderSummaryChip(model);
}

function computeModel(parameters) {
  const reflector = computeReflectorResponse(parameters);
  const concha = computeConchaResponse(parameters);
  const canal = computeCanalResponse(parameters);
  const reflection = computeReflectionResponse(parameters);
  const middle = computeMiddleEarResponse(parameters);

  const combinedMagnitude = FREQUENCIES.map((_, index) => (
    reflector.magnitudes[index] *
    concha.magnitudes[index] *
    canal.magnitudes[index] *
    reflection.magnitudes[index]
  ));

  const outerDb = combinedMagnitude.map(magnitudeToDb);
  const combinedTotalMagnitude = combinedMagnitude.map(
    (magnitude, index) => magnitude * middle.magnitudes[index],
  );
  const combinedTotalDb = combinedTotalMagnitude.map(magnitudeToDb);
  const combinedPeak = findExtremum(FREQUENCIES, combinedTotalDb, "max");
  const highBandNotch = findExtremum(
    FREQUENCIES.filter((frequency) => frequency >= 4000),
    combinedTotalDb.slice(FREQUENCIES.findIndex((frequency) => frequency >= 4000)),
    "min",
  );
  return {
    parameters,
    reflectorDb: reflector.magnitudes.map(magnitudeToDb),
    conchaDb: concha.magnitudes.map(magnitudeToDb),
    canalDb: canal.magnitudes.map(magnitudeToDb),
    reflectionDb: reflection.magnitudes.map(magnitudeToDb),
    combinedDb: combinedTotalDb,
    outerDb,
    middleDb: middle.magnitudes.map(magnitudeToDb),
    middlePhaseDeg: middle.phasesDeg,
    middleInputImpedanceDb: middle.inputImpedanceMagnitudes.map(magnitudeToDb),
    combinedTotalDb,
    metrics: {
      reflector: reflector.metrics,
      concha: concha.metrics,

```

```

    canal: canal.metrics,
    reflection: reflection.metrics,
    middle: middle.metrics,
    combinedPeak,
    highBandNotch,
  },
};
}

function computeReflectorResponse(parameters) {
  const reflectorLength = millimetersToMeters(parameters.reflectorLengthMm);
  const reflectorRadius = millimetersToMeters(parameters.reflectorRadiusMm);
  const ratio = reflectorLength / reflectorRadius;
  const breakHz = SPEED_OF_SOUND / (16 * reflectorRadius);
  const slopeDbPerOctave = clamp(6 * (ratio / 0.5), 3.5, 9.0);
  const maxGainDb = clamp(12 * Math.sqrt(ratio / 0.5), 6, 18);

  const magnitudes = FREQUENCIES.map((frequency) => {
    const octavesAboveBreak = Math.max(0, Math.log2(frequency / breakHz));
    const gainDb = Math.min(maxGainDb, slopeDbPerOctave * octavesAboveBreak);
    return dbToMagnitude(gainDb);
  });

  return {
    magnitudes,
    metrics: {
      breakHz,
      ratio,
      slopeDbPerOctave,
      maxGainDb,
    },
  };
}

function computeConchaResponse(parameters) {
  const volume = cubicCentimetersToCubicMeters(parameters.conchaVolumeCc);
  const radius = millimetersToMeters(parameters.conchaApertureMm);
  const apertureArea = Math.PI * radius ** 2;
  const depth = millimetersToMeters(parameters.conchaDepthMm);
  const effectiveLength = depth + 1.7 * radius;
  const helmholtzHz =
    (SPEED_OF_SOUND / (2 * Math.PI)) *
    Math.sqrt(apertureArea / (volume * effectiveLength));

  const resonanceIndex = Math.sqrt(apertureArea / (volume * effectiveLength));
  const q = clamp(0.7 + 2.2 * (radius / Math.max(depth, 0.001)), 0.7, 3.1);
  const gainDb = clamp(5 + 4 * Math.log2(resonanceIndex / 32), 2, 12);
  const filter = makePeakingFilter(helmholtzHz, q, gainDb);
  const magnitudes = FREQUENCIES.map((frequency) =>
    biquadMagnitudeAtFrequency(filter, frequency),
  );

  return {
    magnitudes,
    metrics: {
      helmholtzHz,
      effectiveLength,
      apertureArea,
      q,
      gainDb,
    },
  };
}

function computeCanalResponse(parameters) {
  const radius = millimetersToMeters(parameters.canalDiameterMm / 2);
  const length = millimetersToMeters(parameters.canalLengthMm);
  const effectiveLength = length + 0.6 * radius;
  const f1 = SPEED_OF_SOUND / (4 * effectiveLength);
  const f2 = 2 * f1;
  const f3 = 3 * f1;

  const narrowness = clamp(Math.sqrt(7 / parameters.canalDiameterMm), 0.7, 1.55);
  const q1 = clamp(1.25 * narrowness, 0.85, 2.3);
  const q2 = clamp(1.05 * narrowness, 0.8, 2.1);
  const q3 = clamp(1.7 * narrowness, 1.0, 2.8);
  const gain1Db = clamp(6.5 * narrowness, 4, 11);
  const notchDepthDb = clamp(4.3 * narrowness, 2.5, 8);

```

```

const gain3Db = clamp(3.2 * narrowness, 1.5, 6.5);

const filters = [
  makePeakingFilter(f1, q1, gain1Db),
  makeNotchFilter(f2, q2, notchDepthDb),
  makePeakingFilter(f3, q3, gain3Db),
];

if (parameters.iec318Enabled) {
  filters.push(makePeakingFilter(220, 0.85, 1.4));
  filters.push(makePeakingFilter(2000, 1.05, 2.3));
}

const magnitudes = FREQUENCIES.map((frequency) =>
  filters.reduce(
    (product, filter) => product * biquadMagnitudeAtFrequency(filter, frequency),
    1,
  ),
);

return {
  magnitudes,
  metrics: {
    effectiveLength,
    f1,
    f2,
    f3,
    q1,
    q2,
    q3,
    gain1Db,
    notchDepthDb,
    gain3Db,
  },
};
}

function computeReflectionResponse(parameters) {
  const direction = deriveDirection(parameters);
  const effectivePathMeters = millimetersToMeters(direction.effectivePathMm);
  const effectiveStrength = direction.effectiveStrength;
  const absorptionCornerHz = 12000;
  const notchHz = SPEED_OF_SOUND / (2 * effectivePathMeters);

  const magnitudes = FREQUENCIES.map((frequency) => {
    const attenuation =
      effectiveStrength / Math.sqrt(1 + (frequency / absorptionCornerHz) ** 2);
    const phase = (2 * Math.PI * frequency * effectivePathMeters) / SPEED_OF_SOUND;
    const real = 1 + attenuation * Math.cos(phase);
    const imaginary = -attenuation * Math.sin(phase);
    return Math.sqrt(real ** 2 + imaginary ** 2);
  });

  return {
    magnitudes,
    metrics: {
      effectivePathMm: direction.effectivePathMm,
      effectiveStrength,
      notchHz,
    },
  };
}

function deriveDirection(parameters) {
  const azimuth = degreesToRadians(parameters.azimuthDeg);
  const elevation = degreesToRadians(parameters.elevationDeg);
  const directionScale =
    1 +
    0.22 * Math.sin(azimuth) +
    0.18 * Math.sin(elevation) -
    0.08 * (1 - Math.cos(azimuth) * Math.cos(elevation));

  const effectivePathMm = clamp(parameters.reflectionPathMm * directionScale, 8, 24);
  const effectiveStrength = clamp(
    parameters.reflectionStrength *
    (1 +
      0.22 * Math.abs(Math.sin(azimuth)) +
      0.12 * Math.max(0, Math.sin(elevation))),

```

```

    0.05,
    0.98,
  );

  return { effectivePathMm, effectiveStrength };
}

// The middle-ear network follows the class diagram and uses normalized model units.
function computeMiddleEarResponse(parameters) {
  const seriesImpedances = FREQUENCIES.map((frequency) => {
    const cavity = Z_cavity(frequency, parameters);
    const eardrumLosses = Z_eardrum(frequency, parameters);
    const ossicles = Z_ossicles(frequency, parameters);
    const joint = Z_joint(frequency, parameters);
    const cochlea = Z_cochlea(frequency, parameters);
    const distalLoad = parallelImpedance(joint, cochlea);
    const forwardPath = addComplex(ossicles, distalLoad);
    const proximalNode = parallelImpedance(eardrumLosses, forwardPath);
    const input = addComplex(cavity, proximalNode);
    const source = complex(MIDDLE_CIRCUIT_CONSTANTS.sourceImpedance, 0);
    const proximalDivider = divideComplex(
      proximalNode,
      addComplex(source, input),
    );
    const distalDivider = divideComplex(distalLoad, forwardPath);
    const transmission = scaleComplex(
      multiplyComplex(proximalDivider, distalDivider),
      MIDDLE_CIRCUIT_CONSTANTS.transmissionScale,
    );

    return { input, transmission };
  });

  const magnitudes = seriesImpedances.map(({ transmission }) => complexMagnitude(transmission));
  const phasesDeg = seriesImpedances.map(({ transmission }) =>
    (Math.atan2(transmission.imaginary, transmission.real) * 180) / Math.PI,
  );
  const inputImpedanceMagnitudes = seriesImpedances.map(({ input }) => complexMagnitude(input));
  const transmissionDb = magnitudes.map(magnitudeToDb);
  const inputImpedanceDb = inputImpedanceMagnitudes.map(magnitudeToDb);

  return {
    magnitudes,
    phasesDeg,
    inputImpedanceMagnitudes,
    metrics: {
      transmissionPeak: findExtremum(FREQUENCIES, transmissionDb, "max"),
      inputImpedancePeak: findExtremum(FREQUENCIES, inputImpedanceDb, "max"),
    },
  };
}

function Z_cavity(frequency, parameters) {
  const omega = 2 * Math.PI * frequency;
  const tympanicCompliance = parameters.middleCavityVolumeCc * 0.000002;
  const passageBranch = addComplex(
    impedanceCompliance(
      omega,
      MIDDLE_CIRCUIT_CONSTANTS.cavityPassageCompliance,
    ),
    addComplex(
      impedanceInertance(
        omega,
        MIDDLE_CIRCUIT_CONSTANTS.cavityPassageInertance,
      ),
      complex(MIDDLE_CIRCUIT_CONSTANTS.cavityPassageResistance, 0),
    ),
  );
  const wallLossBranch = complex(MIDDLE_CIRCUIT_CONSTANTS.cavityWallResistance, 0);
  const tympanicBranch = impedanceCompliance(omega, tympanicCompliance);
  return parallelImpedance(
    parallelImpedance(passageBranch, wallLossBranch),
    tympanicBranch,
  );
}

function Z_eardrum(frequency, parameters) {
  const omega = 2 * Math.PI * frequency;

```

```

const secondMode = addComplex(
  impedanceCompliance(
    omega,
    MIDDLE_CIRCUIT_CONSTANTS.eardrumSecondaryCompliance,
  ),
  complex(MIDDLE_CIRCUIT_CONSTANTS.eardrumSecondaryResistance, 0),
);
const uncoupledMassBranch = impedanceInertance(
  omega,
  parameters.middleInertance * 0.8,
);
return addComplex(
  impedanceCompliance(omega, parameters.middleCompliance * 1.2),
  addComplex(
    parallelImpedance(uncoupledMassBranch, secondMode),
    complex(parameters.middleResistance * 0.6, 0),
  ),
);
}

function Z_ossicles(frequency, parameters) {
  const omega = 2 * Math.PI * frequency;
  return addComplex(
    impedanceCompliance(omega, parameters.middleCompliance * 0.7),
    addComplex(
      impedanceInertance(omega, parameters.middleInertance * 0.4),
      complex(parameters.middleResistance * 0.45, 0),
    ),
  );
}

function Z_joint(frequency, parameters) {
  const omega = 2 * Math.PI * frequency;
  return addComplex(
    impedanceCompliance(omega, parameters.jointCompliance),
    complex(parameters.middleResistance * 0.35, 0),
  );
}

function Z_cochlea(frequency, parameters) {
  const omega = 2 * Math.PI * frequency;
  return addComplex(
    impedanceCompliance(omega, MIDDLE_CIRCUIT_CONSTANTS.cochlearCompliance),
    addComplex(
      impedanceInertance(
        omega,
        MIDDLE_CIRCUIT_CONSTANTS.cochlearInertance,
      ),
      complex(parameters.cochlearLoad, 0),
    ),
  );
}

function impedanceInertance(omega, inertance) {
  return complex(0, omega * inertance);
}

function impedanceCompliance(omega, compliance) {
  return complex(0, -1 / Math.max(omega * compliance, 1e-12));
}

function complex(real, imaginary) {
  return { real, imaginary };
}

function addComplex(first, second) {
  return complex(first.real + second.real, first.imaginary + second.imaginary);
}

function multiplyComplex(first, second) {
  return complex(
    first.real * second.real - first.imaginary * second.imaginary,
    first.real * second.imaginary + first.imaginary * second.real,
  );
}

function divideComplex(numerator, denominator) {
  const denominatorMagnitudeSquared =

```

```

    denominator.real ** 2 + denominator.imaginary ** 2;
return complex(
    (numerator.real * denominator.real + numerator.imaginary * denominator.imaginary) /
    denominatorMagnitudeSquared,
    (numerator.imaginary * denominator.real - numerator.real * denominator.imaginary) /
    denominatorMagnitudeSquared,
);
}

function scaleComplex(value, scalar) {
    return complex(value.real * scalar, value.imaginary * scalar);
}

function complexMagnitude(value) {
    return Math.sqrt(value.real ** 2 + value.imaginary ** 2);
}

function parallelImpedance(first, second) {
    return divideComplex(
        multiplyComplex(first, second),
        addComplex(first, second),
    );
}

function buildSweepSeries(parameters) {
    const key = dom.sweepParameter.value;
    const traceCount = Number(dom.sweepTraceCount.value);
    const definition = PARAMETER_LOOKUP[key];
    const series = [];

    for (let index = 0; index < traceCount; index += 1) {
        const ratio = traceCount === 1 ? 0 : index / (traceCount - 1);
        const sweptValue = interpolate(definition.min, definition.max, ratio);
        const sweptParameters = { ...parameters, [key]: sweptValue };
        const model = computeModel(sweptParameters);
        const hue = 190 + index * 18;

        series.push({
            label: `${definition.label}: ${formatParameterValue(key, sweptValue)}`,
            values: model.combinedTotalDb,
            color: `hsl(${hue} 58% ${index === traceCount - 1 ? "42%" : "52%"})`,
        });
    }

    return series;
}

function renderMarkerList(model) {
    const rows = [
        ["Reflector break", formatFrequency(model.metrics.reflector.breakHz)],
        ["Helmholtz resonance", formatFrequency(model.metrics.concha.helmholtzHz)],
        ["Canal mode n = 1", formatFrequency(model.metrics.canal.f1)],
        ["Canal antiresonance n = 2", formatFrequency(model.metrics.canal.f2)],
        ["Canal mode n = 3", formatFrequency(model.metrics.canal.f3)],
        ["Pinna notch", formatFrequency(model.metrics.reflection.notchHz)],
        [
            "Effective reflection path",
            `${model.metrics.reflection.effectivePathMm.toFixed(1)} mm`,
        ],
        [
            "Combined response peak",
            `${formatFrequency(model.metrics.combinedPeak.frequency)} @ ${formatDb(model.metrics.combinedPeak.value)}`,
        ],
        [
            "Middle-ear transmission peak",
            `${formatFrequency(model.metrics.middle.transmissionPeak.frequency)} @ ${formatDb(model.metrics.middle.transmissionPeak.value)}`,
        ],
        [
            "Input impedance peak",
            formatFrequency(model.metrics.middle.inputImpedancePeak.frequency),
        ],
    ];

    dom.markerList.innerHTML = rows
        .map(
            ([label, value]) => `
                <div class="marker-row">
                    <strong>${label}</strong>
            `
        )
        .join('');

```

```

    <span>${value}</span>
  </div>
  `
)
.join("");
}

```

```

function renderExperiments(parameters, baselineModel) {
  const experiments = [
    {
      key: "middleInertance",
      label: "Inertance +20%",
      prediction: "The resonance should move lower because f0 is proportional to 1/sqrt(LC).",
    },
    {
      key: "middleCompliance",
      label: "Compliance +20%",
      prediction: "The resonance should move lower and the reactive balance should change near the middle-ear peak.",
    },
    {
      key: "middleResistance",
      label: "Resistance +20%",
      prediction: "Loss should increase, reducing the sharpness and usually lowering the peak magnitude.",
    },
  ];
}

```

```

dom.experimentList.innerHTML = experiments
  .map((experiment) => {
    const alteredParameters = {
      ...parameters,
      [experiment.key]: parameters[experiment.key] * 1.2,
    };
    const alteredModel = computeModel(alteredParameters);
    const baselinePeak = baselineModel.metrics.middle.transmissionPeak;
    const alteredPeak = alteredModel.metrics.middle.transmissionPeak;
    const frequencyChange = alteredPeak.frequency - baselinePeak.frequency;
    const levelChange = alteredPeak.value - baselinePeak.value;
    const direction = frequencyChange < -1 ? "lower" : frequencyChange > 1 ? "higher" : "nearly unchanged";

    return `
      <article class="experiment-row">
        <strong>${experiment.label}</strong>
        <p><span>Prediction:</span> ${experiment.prediction}</p>
        <p><span>Model result:</span> peak moved ${direction} to ${formatFrequency(alteredPeak.frequency)} (${formatSignedHz(frequencyChange)}); peak
        level changed ${formatDb(levelChange)}</p>
      </article>
    `;
  })
  .join("");
}

```

```

function renderSummaryChip(model) {
  const peak = model.metrics.combinedPeak;
  const notch = model.metrics.reflection;
  dom.summaryChip.textContent =
    `Total peak ${formatFrequency(peak.frequency)} - middle-ear peak ${formatFrequency(model.metrics.middle.transmissionPeak.frequency)} - pinna notch $
    {formatFrequency(notch.notchHz)}`;
}

```

```

function exportReportSnippet() {
  const model = latestModel;
  if (!model) {
    return;
  }
}

```

```

const parameters = model.parameters;
const lines = [
  "# Outer + Middle Ear Model Summary",
  "",
  `Date: ${new Date().toISOString().slice(0, 10)}`,
  "",
  "## Current anatomical parameters",
  ...PARAMETER_DEFS.map(
    (definition) =>
      `
        - ${definition.label}: ${formatParameterValue(
          definition.key,
          parameters[definition.key],
        )}
      `
  ),
]

```

```

),
`- IEC 318 auxiliary branches: ${parameters.iec318Enabled ? "On" : "Off"}`,
"" ,
"## Calculated frequencies",
`- Reflector break: ${formatFrequency(model.metrics.reflector.breakHz)}`,
`- Concha Helmholtz resonance: ${formatFrequency(model.metrics.concha.helmholtzHz)}`,
`- Ear-canal quarter-wave mode (n=1): ${formatFrequency(model.metrics.canal.f1)}`,
`- Ear-canal antiresonance estimate (n=2): ${formatFrequency(model.metrics.canal.f2)}`,
`- Ear-canal third mode (n=3): ${formatFrequency(model.metrics.canal.f3)}`,
`- Pinna reflection notch: ${formatFrequency(model.metrics.reflection.notchHz)}`,
`- Effective reflection path: ${model.metrics.reflection.effectivePathMm.toFixed(1)} mm`,
`- Middle-ear transmission peak: ${formatFrequency(model.metrics.middle.transmissionPeak.frequency)} @ $
formatDb(model.metrics.middle.transmissionPeak.value)},
`- Middle-ear input impedance peak: ${formatFrequency(model.metrics.middle.inputImpedancePeak.frequency)}`,
"" ,
"## Equations used",
"- f_break = c / (16a)",
"- f_H = c / 2pi * sqrt(A / (V L_eff))",
"- f_n = n c / (4 L_eff)",
"- f_notch = c / (2 r_eff)",
"- Z_R = R",
"- Z_L = j omega L",
"- Z_C = 1 / (j omega C)",
"- Z_cavity = (Z_CP + Z_LA + R_A) || R_M || Z_CT",
"- Z_eardrum = Z_CD1 + (Z_LD || (Z_CD2 + R_D2)) + R_D1",
"- Z_ossicles = Z_CO + Z_LO + R_O",
"- Z_distal = Z_joint || Z_cochlea",
"- Z_in = Z_cavity + (Z_eardrum || (Z_ossicles + Z_distal))",
"- H_total = H_outer x H_middle",
"" ,
"## Middle-ear topology",
"- The cavity block is a series block with parallel C_P-L_A-R_A, R_M, and C_T branches.",
"- The eardrum-loss block is a shunt branch, followed by series C_O-L_O-R_O",
"- Joint and stapes/cochlear branches are parallel shunt loads at the distal node.",
"- The cochlear branch is only a terminating load; inner-ear mechanics are not modeled.",
"- The transfer uses an ideal-source divider across the cavity/proximal network and a distal-node divider into the cochlear-load branch.",
"- Transmission is unit-normalized; no source resistance or arbitrary gain is included because neither appears in the class circuit.",
"- The outer and middle models are cascaded without pressure-loading feedback from the middle ear into the outer-ear filters.",
"" ,
"## Brief discussion",
"- Increasing ear-canal length lowers the quarter-wave resonances.",
"- Increasing concha aperture or decreasing concha volume raises the Helmholtz resonance.",
"- Larger effective reflection paths push the pinna notch downward in frequency.",
"- The combined response is the product of the outer-ear response and middle-ear transmission, so middle-ear mass/compliance/loss changes reshape the
total response.",
"" ,
"## AI-assisted experiments",
...buildExperimentReportLines(parameters, model),
"" ,
"## Parameter provenance",
"- Canal length, concha dimensions, and cavity volume are physically interpretable anatomical parameters, but the baseline values are assumed population-
representative values.",
"- Middle-ear inrtance, compliance, resistance, and cochlear load are normalized model parameters; they are assumed/fitted placeholders rather than
patient measurements.",
"" ,
"## Model limitations",
"- The middle-ear circuit follows the class topology, but its normalized component values are not patient-specific measurements.",
"- The cascade assumes no loading feedback between the middle-ear input impedance and outer-ear transfer function.",
"- The model is not subject-specific and does not represent nonlinear tissue behavior, three-dimensional radiation, or individual HRTFs.",
"" ,
"## Copilot note",
"- This interactive model and the summary text were built with GPT-5.6 in Codex as a coding copilot. I rejected the earlier cavity-shunt/all-series
interpretation after checking the class diagram node by node, then reran the model with the corrected topology.",
"" ,
];

const blob = new Blob([lines.join("\n")], { type: "text/markdown" });
const url = URL.createObjectURL(blob);
const link = document.createElement("a");
link.href = url;
link.download = "outer-ear-model-summary.md";
link.click();
URL.revokeObjectURL(url);
}

function buildExperimentReportLines(parameters, baselineModel) {
return [
["middleInrtance", "Eardrum + ossicle inrtance"],

```

```

["middleCompliance", "Eardrum compliance"],
["middleResistance", "Eardrum/joint resistance"],
].flatMap(([,key, label]) => {
  const alteredModel = computeModel({
    ...parameters,
    [key]: parameters[key] * 1.2,
  });
  const baselinePeak = baselineModel.metrics.middle.transmissionPeak;
  const alteredPeak = alteredModel.metrics.middle.transmissionPeak;
  return [
    ` - ${label} +20%: peak changed from ${formatFrequency(baselinePeak.frequency)} to ${formatFrequency(alteredPeak.frequency)}; peak level changed $
    {formatDb(alteredPeak.value - baselinePeak.value)}.`,
  ];
});
}

```

```

function renderPlot(svg, series, options = {}) {
  const [width, height] = svg
    .getAttribute("viewBox")
    .split(" ")
    .slice(2)
    .map(Number);
  const padding = { top: 16, right: 18, bottom: 34, left: 54 };
  const innerWidth = width - padding.left - padding.right;
  const innerHeight = height - padding.top - padding.bottom;
  const yRange = options.yRange || DB_RANGE;

  svg.innerHTML = "";

  const background = createSvgElement("rect", {
    x: 0,
    y: 0,
    width,
    height,
    rx: 16,
    fill: "transparent",
  });
  svg.appendChild(background);

  const yTicks = options.yTicks || [24, 18, 12, 6, 0, -6, -12, -18];
  yTicks.forEach((tick) => {
    const y = padding.top + innerHeight * (1 - (tick - yRange.min) / (yRange.max - yRange.min));
    svg.appendChild(
      createSvgElement("line", {
        x1: padding.left,
        y1: y,
        x2: width - padding.right,
        y2: y,
        stroke: "var(--plot-grid)",
        "stroke-width": 1,
      })
    );
    const label = createSvgElement("text", {
      x: padding.left - 10,
      y: y + 4,
      "text-anchor": "end",
      class: "tick-label",
    });
    label.textContent = `${tick}`;
    svg.appendChild(label);
  });
}

```

```

const frequencyTicks = [20, 50, 100, 200, 500, 1000, 2000, 5000, 10 000, 20 000];
frequencyTicks.forEach((tick) => {
  const x = padding.left + innerWidth * normalizeLogFrequency(tick);
  svg.appendChild(
    createSvgElement("line", {
      x1: x,
      y1: padding.top,
      x2: x,
      y2: height - padding.bottom,
      stroke: "var(--plot-grid)",
      "stroke-width": 1,
    })
  );
  const label = createSvgElement("text", {
    x,
    y: height - 12,

```

```

    "text-anchor": "middle",
    class: "tick-label",
  });
  label.textContent = formatFrequencyTick(tick);
  svg.appendChild(label);
});

svg.appendChild(
  createSvgElement("rect", {
    x: padding.left,
    y: padding.top,
    width: innerWidth,
    height: innerHeight,
    fill: "none",
    stroke: "rgba(45, 32, 22, 0.18)",
    "stroke-width": 1.2,
    rx: 12,
  }),
);

if (options.markers) {
  options.markers.forEach((marker, index) => {
    const x = padding.left + innerWidth * normalizeLogFrequency(marker.frequency);
    svg.appendChild(
      createSvgElement("line", {
        x1: x,
        y1: padding.top,
        x2: x,
        y2: height - padding.bottom,
        stroke: index % 2 === 0 ? "rgba(42, 127, 122, 0.45)" : "rgba(213, 103, 63, 0.45)",
        "stroke-width": 1.2,
        "stroke-dasharray": "4 4",
      }),
    );
    const label = createSvgElement("text", {
      x,
      y: padding.top + 12 + index * 12,
      "text-anchor": "middle",
      class: "marker-label",
    });
    label.textContent = marker.label;
    svg.appendChild(label);
  });
}

series.forEach((entry) => {
  const pathData = entry.values
    .map((value, index) => {
      const x = padding.left + innerWidth * normalizeLogFrequency(FREQUENCIES[index]);
      const y =
        padding.top +
        innerHeight * (1 - (value - yRange.min) / (yRange.max - yRange.min));
      return `${index === 0 ? "M" : "L"} ${x.toFixed(2)} ${clamp(y, padding.top, height - padding.bottom).toFixed(2)}`;
    })
    .join(" ");

  svg.appendChild(
    createSvgElement("path", {
      d: pathData,
      fill: "none",
      stroke: entry.color,
      "stroke-width": entry.label === "Combined" ? 3.4 : 2.4,
      "stroke-linecap": "round",
      "stroke-linejoin": "round",
      opacity: options.showLegend ? 0.9 : 1,
    }),
  );
});

const xLabel = createSvgElement("text", {
  x: padding.left + innerWidth / 2,
  y: height - 4,
  "text-anchor": "middle",
  class: "axis-label",
});
xLabel.textContent = "Frequency (Hz)";
svg.appendChild(xLabel);

```

```

const yLabel = createSvgElement("text", {
  x: 16,
  y: padding.top + innerHeight / 2,
  transform: `rotate(-90 16 ${padding.top + innerHeight / 2})`,
  "text-anchor": "middle",
  class: "axis-label",
});
yLabel.textContent = options.yLabel || "Magnitude (dB)";
svg.appendChild(yLabel);

if (options.showLegend) {
  const legendGroup = createSvgElement("g", {});
  series.forEach((entry, index) => {
    const x = padding.left + 10;
    const y = padding.top + 14 + index * 14;
    legendGroup.appendChild(
      createSvgElement("line", {
        x1: x,
        y1: y,
        x2: x + 18,
        y2: y,
        stroke: entry.color,
        "stroke-width": 3,
        "stroke-linecap": "round",
      }),
    );
    const label = createSvgElement("text", {
      x: x + 24,
      y: y + 4,
      class: "legend-label",
    });
    label.textContent = entry.label;
    legendGroup.appendChild(label);
  });
  svg.appendChild(legendGroup);
}

function playNoisePreview() {
  const context = ensureAudioContext();
  stopAudio();
  const buffer = context.createBuffer(1, context.sampleRate * 2, context.sampleRate);
  const samples = buffer.getChannelData(0);
  for (let index = 0; index < samples.length; index += 1) {
    samples[index] = Math.random() * 2 - 1;
  }

  const source = context.createBufferSource();
  source.buffer = buffer;
  source.loop = true;
  connectModelledAudio(source, readParameters());
  source.start();
  audioState.source = source;
  setAudioStatus("Playing looped noise through the current outer-ear model.");
}

function playToneSweepPreview() {
  const context = ensureAudioContext();
  stopAudio();
  const oscillator = new OscillatorNode(context, { type: "sine", frequency: 180 });
  oscillator.frequency.exponentialRampToValueAtTime(16000, context.currentTime + 5);
  connectModelledAudio(oscillator, readParameters());
  oscillator.start();
  oscillator.stop(context.currentTime + 5);
  oscillator.addEventListener("ended", () => {
    cleanupAudioGraph();
    setAudioStatus("Sweep finished.");
  });
  audioState.source = oscillator;
  setAudioStatus("Playing a logarithmic sweep through the current outer-ear model.");
}

async function handleAudioUpload(event) {
  const file = event.target.files?.[0];
  if (!file) {
    audioState.uploadedBuffer = null;
    dom.playFileButton.disabled = true;
    setAudioStatus("Audio upload cleared.");
  }
}

```

```

    return;
  }

  try {
    const context = ensureAudioContext();
    const arrayBuffer = await file.arrayBuffer();
    audioState.uploadedBuffer = await context.decodeAudioData(arrayBuffer.slice(0));
    dom.playFileButton.disabled = false;
    setAudioStatus(`Loaded "${file.name}". Ready to preview.`);
  } catch (error) {
    audioState.uploadedBuffer = null;
    dom.playFileButton.disabled = true;
    setAudioStatus("Could not decode that audio file.");
  }
}

function playUploadedAudio() {
  if (!audioState.uploadedBuffer) {
    return;
  }

  const context = ensureAudioContext();
  stopAudio();
  const source = context.createBufferSource();
  source.buffer = audioState.uploadedBuffer;
  connectModelledAudio(source, readParameters());
  source.start();
  source.addEventListener("ended", () => {
    cleanupAudioGraph();
    setAudioStatus("Uploaded audio finished.");
  });
  audioState.source = source;
  setAudioStatus("Playing the uploaded audio through the current outer-ear model.");
}

function connectModelledAudio(sourceNode, parameters) {
  const context = ensureAudioContext();
  const model = computeModel(parameters);
  const reflector = model.metrics.reflector;
  const concha = model.metrics.concha;
  const canal = model.metrics.canal;
  const reflection = model.metrics.reflection;

  const shelf = new BiquadFilterNode(context, {
    type: "highshelf",
    frequency: clamp(reflector.breakHz, 80, context.sampleRate * 0.45),
    gain: reflector.maxGainDb * 0.7,
  });
  const conchaPeak = new BiquadFilterNode(context, {
    type: "peaking",
    frequency: clamp(concha.helmholtzHz, 80, context.sampleRate * 0.45),
    Q: concha.q,
    gain: concha.gainDb,
  });
  const canalPeak = new BiquadFilterNode(context, {
    type: "peaking",
    frequency: clamp(canal.f1, 80, context.sampleRate * 0.45),
    Q: canal.q1,
    gain: canal.gain1Db,
  });
  const canalNotch = new BiquadFilterNode(context, {
    type: "notch",
    frequency: clamp(canal.f2, 80, context.sampleRate * 0.45),
    Q: canal.q2,
  });
  const canalPeak3 = new BiquadFilterNode(context, {
    type: "peaking",
    frequency: clamp(canal.f3, 80, context.sampleRate * 0.45),
    Q: canal.q3,
    gain: canal.gain3Db,
  });
  const mainGain = new GainNode(context, { gain: 0.52 });
  const dryGain = new GainNode(context, { gain: 1 });
  const delayNode = new DelayNode(context, {
    delayTime: clamp(
      millimetersToMeters(reflection.effectivePathMm) / SPEED_OF_SOUND,
      0,
      0.08,
    ),
  });

```

```

    ),
    maxDelayTime: 0.1,
  });
  const wetGain = new GainNode(context, {
    gain: reflection.effectiveStrength * 0.62,
  });
  const lowpass = new BiquadFilterNode(context, {
    type: "lowpass",
    frequency: 12000,
    Q: 0.7,
  });

  const chainNodes = [sourceNode, shelf, conchaPeak, canalPeak, canalNotch, canalPeak3];
  if (parameters.iec318Enabled) {
    chainNodes.push(
      new BiquadFilterNode(context, {
        type: "peaking",
        frequency: 220,
        Q: 0.85,
        gain: 1.4,
      }),
      new BiquadFilterNode(context, {
        type: "peaking",
        frequency: 2000,
        Q: 1.05,
        gain: 2.3,
      })
    );
  }

  for (let index = 0; index < chainNodes.length - 1; index += 1) {
    chainNodes[index].connect(chainNodes[index + 1]);
  }

  const outputNode = chainNodes[chainNodes.length - 1];
  outputNode.connect(dryGain);
  outputNode.connect(delayNode);
  delayNode.connect(lowpass);
  lowpass.connect(wetGain);
  dryGain.connect(mainGain);
  wetGain.connect(mainGain);
  mainGain.connect(context.destination);

  audioState.sourceNodes = [sourceNode];
  audioState.activeNodes = [
    shelf,
    conchaPeak,
    canalPeak,
    canalNotch,
    canalPeak3,
    dryGain,
    delayNode,
    wetGain,
    lowpass,
    mainGain,
    ...chainNodes.slice(6),
  ];
}

function stopAudio() {
  if (audioState.source) {
    try {
      audioState.source.stop();
    } catch (error) {
      // Source may already be stopped.
    }
  }
  cleanupAudioGraph();
  setAudioStatus("Audio idle.");
}

function cleanupAudioGraph() {
  if (audioState.source) {
    try {
      audioState.source.disconnect();
    } catch (error) {
      // Ignore disconnect errors during cleanup.
    }
  }
}

```

```

    }
    audioState.activeNodes.forEach((node) => {
      try {
        node.disconnect();
      } catch (error) {
        // Ignore disconnect errors during cleanup.
      }
    });
    audioState.source = null;
    audioState.sourceNodes = [];
    audioState.activeNodes = [];
  }

function ensureAudioContext() {
  if (!audioState.context) {
    audioState.context = new (window.AudioContext || window.webkitAudioContext)();
  }
  if (audioState.context.state === "suspended") {
    audioState.context.resume();
  }
  return audioState.context;
}

function setAudioStatus(message) {
  dom.audioStatus.textContent = message;
}

function makePeakingFilter(frequency, q, gainDb) {
  const omega = (2 * Math.PI * frequency) / ANALYSIS_SAMPLE_RATE;
  const alpha = Math.sin(omega) / (2 * q);
  const amplitude = Math.pow(10, gainDb / 40);
  const cosine = Math.cos(omega);

  const b0 = 1 + alpha * amplitude;
  const b1 = -2 * cosine;
  const b2 = 1 - alpha * amplitude;
  const a0 = 1 + alpha / amplitude;
  const a1 = -2 * cosine;
  const a2 = 1 - alpha / amplitude;

  return normalizeBiquad({ b0, b1, b2, a0, a1, a2 });
}

function makeNotchFilter(frequency, q, depthDb) {
  const omega = (2 * Math.PI * frequency) / ANALYSIS_SAMPLE_RATE;
  const alpha = Math.sin(omega) / (2 * q);
  const cosine = Math.cos(omega);
  const notch = normalizeBiquad({
    b0: 1,
    b1: -2 * cosine,
    b2: 1,
    a0: 1 + alpha,
    a1: -2 * cosine,
    a2: 1 - alpha,
  });

  notch.mix = clamp(depthDb / 12, 0.2, 0.85);
  return notch;
}

function normalizeBiquad(coefficients) {
  return {
    b0: coefficients.b0 / coefficients.a0,
    b1: coefficients.b1 / coefficients.a0,
    b2: coefficients.b2 / coefficients.a0,
    a1: coefficients.a1 / coefficients.a0,
    a2: coefficients.a2 / coefficients.a0,
    mix: coefficients.mix ?? 1,
  };
}

function biquadMagnitudeAtFrequency(filter, frequency) {
  const omega = (2 * Math.PI * frequency) / ANALYSIS_SAMPLE_RATE;
  const cos1 = Math.cos(omega);
  const sin1 = Math.sin(omega);
  const cos2 = Math.cos(2 * omega);
  const sin2 = Math.sin(2 * omega);

```

```

const numeratorReal = filter.b0 + filter.b1 * cos1 + filter.b2 * cos2;
const numeratorImaginary = -filter.b1 * sin1 - filter.b2 * sin2;
const denominatorReal = 1 + filter.a1 * cos1 + filter.a2 * cos2;
const denominatorImaginary = -filter.a1 * sin1 - filter.a2 * sin2;

const numeratorMagnitude = Math.sqrt(
  numeratorReal ** 2 + numeratorImaginary ** 2,
);
const denominatorMagnitude = Math.sqrt(
  denominatorReal ** 2 + denominatorImaginary ** 2,
);
const rawMagnitude = numeratorMagnitude / denominatorMagnitude;

if (filter.mix === 1) {
  return rawMagnitude;
}

return (1 - filter.mix) + filter.mix * rawMagnitude;
}

function findExtremum(frequencies, values, mode) {
  let bestIndex = 0;
  let bestValue = values[0];

  values.forEach((value, index) => {
    if ((mode === "max" && value > bestValue) || (mode === "min" && value < bestValue)) {
      bestIndex = index;
      bestValue = value;
    }
  });

  return { frequency: frequencies[bestIndex], value: bestValue };
}

function formatParameterValue(key, value) {
  switch (key) {
    case "conchaVolumeCc":
    case "middleCavityVolumeCc":
      return `${value.toFixed(1)} cc`;
    case "reflectionStrength":
      return value.toFixed(2);
    case "azimuthDeg":
    case "elevationDeg":
      return `${value.toFixed(0)} deg`;
    case "middleInertance":
    case "middleCompliance":
    case "jointCompliance":
      return `${value.toFixed(6)} m.u.`;
    case "middleResistance":
    case "cochlearLoad":
      return `${value.toFixed(2)} m.u.`;
    default:
      return `${value.toFixed(1)} mm`;
  }
}

function formatFrequency(frequency) {
  if (frequency >= 1000) {
    const precision = frequency < 10 000 ? 2 : 1;
    return `${(frequency / 1000).toFixed(precision)} kHz`;
  }
  return `${frequency.toFixed(0)} Hz`;
}

function formatFrequencyTick(frequency) {
  if (frequency >= 1000) {
    return `${frequency / 1000}k`;
  }
  return `${frequency}`;
}

function formatDb(value) {
  return `${value >= 0 ? "+" : ""}${value.toFixed(1)} dB`;
}

function formatSignedHz(value) {
  return `${value >= 0 ? "+" : ""}${value.toFixed(0)} Hz`;
}

```

```

function logSpace(minimum, maximum, count) {
  const output = [];
  const start = Math.log10(minimum);
  const end = Math.log10(maximum);

  for (let index = 0; index < count; index += 1) {
    const ratio = index / (count - 1);
    output.push(10 ** interpolate(start, end, ratio));
  }

  return output;
}

function normalizeLogFrequency(frequency) {
  return (
    (Math.log10(frequency) - Math.log10(FREQUENCIES[0])) /
    (Math.log10(FREQUENCIES[FREQUENCIES.length - 1]) - Math.log10(FREQUENCIES[0]))
  );
}

function createSvgElement(tagName, attributes) {
  const element = document.createElementNS("http://www.w3.org/2000/svg", tagName);
  Object.entries(attributes).forEach(([key, value]) => {
    element.setAttribute(key, value);
  });
  return element;
}

function getInput(key) {
  return document.querySelector(`[data-param="${key}"]`);
}

function dbToMagnitude(value) {
  return 10 ** (value / 20);
}

function magnitudeToDb(value) {
  return 20 * Math.log10(Math.max(value, 1e-6));
}

function millimetersToMeters(value) {
  return value / 1000;
}

function cubicCentimetersToCubicMeters(value) {
  return value * 1e-6;
}

function degreesToRadians(value) {
  return (value * Math.PI) / 180;
}

function interpolate(start, end, ratio) {
  return start + (end - start) * ratio;
}

function clamp(value, minimum, maximum) {
  return Math.min(maximum, Math.max(minimum, value));
}

```